A logical time represents a single point on the HLA time axis. A logical time interval represents the distance between two points on the HLA time axis. During a federation execution, logical time may play two roles. Federates may associate themselves with points on the HLA time axis, and they may associate some of their activities, such as updating an instance attribute's value or sending an interaction, with points on the HLA time axis. These points on the HLA time axis are referred to as time positions when associated with a federate and timestamps when associated with their activities. As a federation execution progresses, federates may advance their time positions along the HLA time axis. Logical time intervals are used to specify a federate's lookahead and by the RTI to perform arithmetic operations with logical time values.

The specific strategy used to advance time in a simulation is driven by the simulation's purpose. For example, faster-than-real time, event-stepped simulations are frequently used in analysis applications, whereas real time, time-stepped simulations are often used in training applications. The strategy chosen for advancing time is an important consideration in the design of a simulation, because it affects both simulation performance and the ability of a simulation to interoperate with other simulations in a federation. For this reason, it is important to document how a simulation supports the advancement of time in a federate's SOM and relevant SOM modules.

IEEE Std 1516.1-2025 provides a variety of time management services to control the advancement of federates along the HLA time axis. These services allow multiple federates with differing internal time advancement strategies to interoperate in a meaningful way. For this reason, it is vital for all federation members to agree on how logical times and logical time intervals are represented across the federation, and document this agreement in the federation's FOM and relevant FOM modules.

During federation execution, logical times and logical time intervals are represented as instances of the time representation abstract data type (TRADT). This abstract data type is provided to the RTI when federation execution is created so that an RTI knows how to use the logical times and logical time intervals. This table allows the data type and semantics of the abstract data type for logical times and logical time intervals to be explained in FOMs, FOM modules, SOMs, and SOM modules.

It is also important to define the lookahead characteristics of both federates and federations. At the individual federate level, the specific means by which lookahead is supported may be a factor in assessing compatibility with other potential federates for a given federation application. For federations, it is important to document a common set of characteristics for how lookahead will be supported across the full federation. Thus, SOMs, SOM modules, FOMs and FOM modules all include representations of lookahead as well as timestamps.

An abstract data type for lookahead is supplied when a federation execution is created. The lookahead entry in this table allows the data type and semantics of this abstract data type to be documented in FOMs, SOMs, FOM modules, and SOM modules.

Semantic differences exist between the way time is represented for the purpose of depicting timestamps versus calculating lookahead. When depicting timestamps, time can be considered to be an absolute value on the HLA time axis, and thus, time comparisons can be drawn to determine if one timestamp is greater than another. Lookahead, in contrast, represents a duration of time, which can be added to timestamps but is generally not used for comparison purposes.

Two time representations, `HLAinteger64Time` and `HLAfloat64Time`, are required to be supported by the HLA and are described in detail in IEEE Std 1516.1-2025. These types shall be provided by all RTIs. Federations may choose to use these default data types, supply their own time representations, or not supply any time representation (e.g., NA).

4.8.2 Table format

The template that shall be used for recording time representation is illustrated in Table 16.

TABLE 16 **Time representation table**

Category	Data type	Semantics
`LogicalTime`	`<type>`	`<semantics>`
`LogicalTimeInterval`	`<type>`	`<semantics>`

The first column (Category) presents the two time-related values that are to be specified in the table: logical time and logical time interval. `LogicalTime` and `LogicalTimeInterval` correspond to the only two rows in this table.

The second column (Data type) shall identify the data type of the time value. This data type shall be chosen from the name of any simple, enumerated, array, fixed record, or variant record data type described in 4.14. "NA" shall be entered in this column when timestamp or lookahead information is inappropriate to the federate or federation.

The third column (Semantics) shall expand and describe the use of the data type for time values. "NA" shall be entered in this column when semantics information is unnecessary.

4.8.3 Inclusion criteria

All federations shall document their use of logical time and logical time interval via the time representation table in a FOM and relevant FOM modules. All federates shall document their representation of logical time via the time representation table in a SOM and relevant SOM modules to the extent that it is used in the federate. For federates that support lookahead, the representation of logical time interval shall be documented via the time representation table in a SOM and relevant SOM modules.

4.8.4 Example

Table 17 provides an example of how the OMT tables are used to describe the representation of time for the restaurant application.

TABLE 17 **Time representation table example**

Category	Data type	Semantics
`LogicalTime`	`HLAinteger64Time`	Integer value expressed in seconds
`LogicalTimeInterval`	`HLAinteger64Time`	Integer value expressed in seconds
Note	NA	

4.9 User-supplied tag table

4.9.1 Purpose/background

The HLA RTI provides a mechanism for federates to supply tags with certain HLA services. These tags can be used to provide additional coordination and control over these services. The user-supplied tag table provides a means of documenting federation agreements regarding the data type to be used with these tags.

TABLE 29 **Basic data representation table**

Name	Size	Interpretation	Endian	Encoding
HLAinteger16BE	16 bit	Integer in the range $[-2^{15}, 2^{15}-1]$	Big	16-bit two's complement signed integer. The most significant bit contains the sign.
HLAinteger32BE	32 bit	Integer in the range $[-2^{31}, 2^{31}-1]$	Big	32-bit two's complement signed integer. The most significant bit contains the sign.
HLAinteger64BE	64 bit	Integer in the range $[-2^{63}, 2^{63}-1]$	Big	64-bit two's complement signed integer. The most significant bit contains the sign.
HLAunsignedInteger16BE	16 bit	Integer in the range $[0, 2^{16}-1]$	Big	16-bit unsigned integer.
HLAunsignedInteger32BE	32 bit	Integer in the range $[0, 2^{32}-1]$	Big	32-bit unsigned integer.
HLAunsignedInteger64BE	64 bit	Integer in the range $[0, 2^{64}-1]$	Big	64-bit unsigned integer.
HLAfloat32BE	32 bit	Single-precision floating point number	Big	32-bit IEEE normalized single-precision format. See IEEE Std 754-1985.
HLAfloat64BE	64 bit	Double-precision floating point number	Big	64-bit IEEE normalized double-precision format. See IEEE Std 754-1985.
HLAoctetPairBE	16 bit	16-bit value	Big	Assumed to be portable among hardware devices.
HLAinteger16LE	16 bit	Integer in the range $[-2^{15}, 2^{15}-1]$	Little	16-bit two's complement signed integer. The most significant bit contains the sign.
HLAinteger32LE	32 bit	Integer in the range $[-2^{31}, 2^{31}-1]$	Little	32-bit two's complement signed integer. The most significant bit contains the sign.
HLAinteger64LE	64 bit	Integer in the range $[-2^{63}, 2^{63}-1]$	Little	64-bit two's complement signed integer. The most significant bit contains the sign.
HLAunsignedInteger8	8 bit	Integer in the range $[0, 2^{8}-1]$	Little	8-bit unsigned integer.
HLAunsignedInteger16LE	16 bit	Integer in the range $[0, 2^{16}-1]$	Little	16-bit unsigned integer.
HLAunsignedInteger32LE	32 bit	Integer in the range $[0, 2^{32}-1]$	Little	32-bit unsigned integer.
HLAunsignedInteger64LE	64 bit	Integer in the range $[0, 2^{64}-1]$	Little	64-bit unsigned integer.
HLAfloat32LE	32 bit	Single-precision floating point number	Little	32-bit IEEE normalized single-precision format. See IEEE Std 754-1985.
HLAfloat64LE	64 bit	Double-precision floating point number	Little	64-bit IEEE normalized double-precision format. See IEEE Std 754-1985.
HLAoctetPairLE	16 bit	16-bit value	Little	Assumed to be portable among hardware devices.
HLAoctet	8 bit	8-bit value	Big	Assumed to be portable among hardware devices.
<name>	<size>	<interpretation>	<endian>	<encoding>
<name>	<size>	<interpretation>	<endian>	<encoding>